Agriculture Support Triage Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Vidhi Gupta	EN22CS3011075
2	Tejashwi Bharti	EN22CS3011036
3	Tanishka Soni	EN22CS3011020
4	Tanish Patidar	EN22CS3011016
5	Shriyansh Shrivastava	EN22CS301986

Group Name: 11D9

Project Number: AAI-11

Industry Mentor Name: Mr. Ashwin

University Mentor Name: Prof. Pradeep Baniya

Academic Year: 2025-26

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features *(Just mention key features here no need to go into details)*
2. Overall Architecture / Workflow
3. Tools & Technologies Used (*If applicable)*

Results & Output

Add the below details here:

1. *Screenshots / outputs*
2. *Reports / dashboards / models*
3. *Key outcomes*

Conclusion

Mention a brief conclusion about your project summing up everything you have worked on and the key learning.

Future Scope & Enhancements

1. Problem Statement & Objectives

1.1 Problem Statement

Traditional agricultural management and Infrastructure as Code (IaC) systems suffer from a "Technical Barrier Gap." Farmers and field supervisors often lack the SQL expertise required to manage structured databases, while DevOps engineers struggle with the manual overhead of provisioning resources for localized agricultural sensors.

The Core Issues are:

- **Data Inaccessibility:** Critical crop data is locked behind structured databases (MySQL) that require specific query languages, making real-time data retrieval difficult for non-technical stakeholders.
- **Static Automation:** Most automation systems are "hard-coded" and cannot adapt to unstructured natural language inputs, such as a user reporting a new pest outbreak or requesting a resource scale-up in a specific region like Punjab.
- **Disconnected Knowledge:** There is a disconnect between structured inventory data (fertilizer quantities) and unstructured expert knowledge (pest control manuals), leading to fragmented decision-making.

1.2 Project Objectives

The primary goal of this project is to bridge the gap between human intent and technical execution using an **Agentic AI Triage System**.

The Specific Objectives are:

- **Implementation of a Multi-Agent Triage System:** To develop a central "Orchestrator" that autonomously classifies user requests and routes them to specialized sub-agents (SQL Agent, RAG Agent, or Provisioning Agent).
- **Natural Language Data Interaction:** To enable users to perform "Zero-UI" database operations (CRUD) on the agriculture_db using plain English or Hindi/Hinglish, handled by an autonomous **NER (Named Entity Recognition) Agent**.
- **Unified Context Retrieval:** To integrate **Retrieval-Augmented Generation (RAG)** using **ChromaDB**, allowing the agent to provide expert agricultural advice by combining real-time database values with embedded documentation.
- **Automated Infrastructure Orchestration:** To demonstrate how Agentic AI can manage containerized environments (Docker) and cloud instances (AWS EC2) based on the dynamic needs of the agricultural project.

1.3 Scope of the Project

The scope encompasses the end-to-end development of a containerized "AgriConnect" platform.

- **Frontend:** A React-based dashboard for user interaction.
- **Backend:** A Flask API serving as the reasoning engine for the AI Agents.
- **Database:** A MySQL instance for structured crop and user management.
- **AI Layer:** Integration of LLMs for autonomous reasoning, intent classification, and parameter extraction.

2. Proposed Solution

The proposed solution is an **AI-Driven IaC Provisioning and Agricultural Management System**. Unlike traditional applications that rely on static forms, this system utilizes a **Multi-Agent Orchestrator**. It acts as an "Intelligent Middleware" that interprets natural language commands—such as "Provision a new database for Wheat in Punjab"—and autonomously executes the necessary SQL commands and infrastructure configurations within a containerized environment.

2.1. Key Features

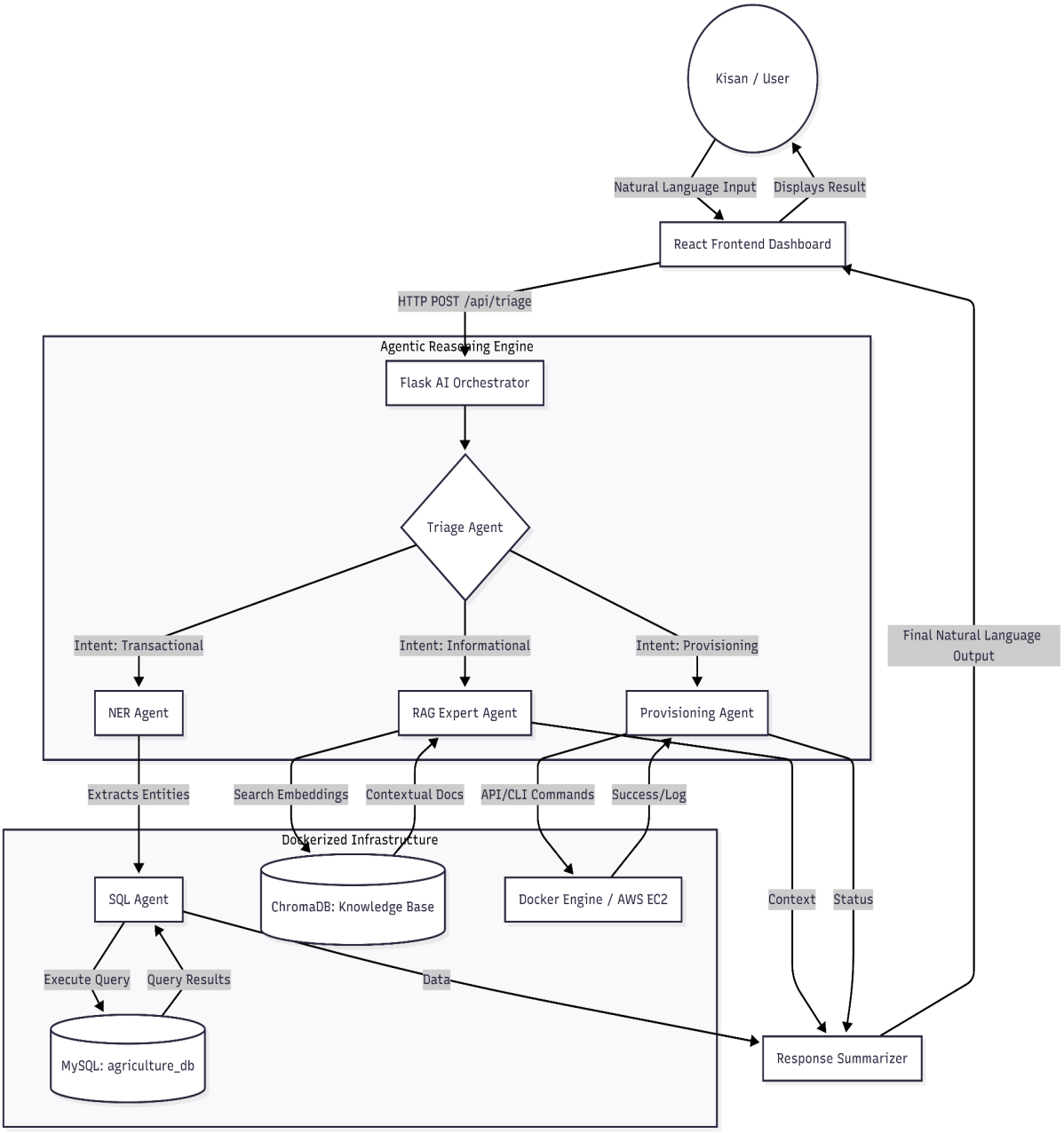
- **Autonomous Triage Agent:** Directs user queries to specialized sub-agents based on intent.
- **NER-Based Parameter Extraction:** Automatically identifies crops, locations, and quantities from unstructured text.
- **Self-Healing Database Integration:** Autonomously manages and updates the agriculture_db using a SQL Agent.
- **RAG-Enhanced Knowledge Retrieval:** Combines real-time database stats with expert agricultural manuals stored in a vector database.
- **Containerized Microservices:** Fully orchestrated environment using Docker for seamless deployment.

2.2 Overall Architecture / Workflow

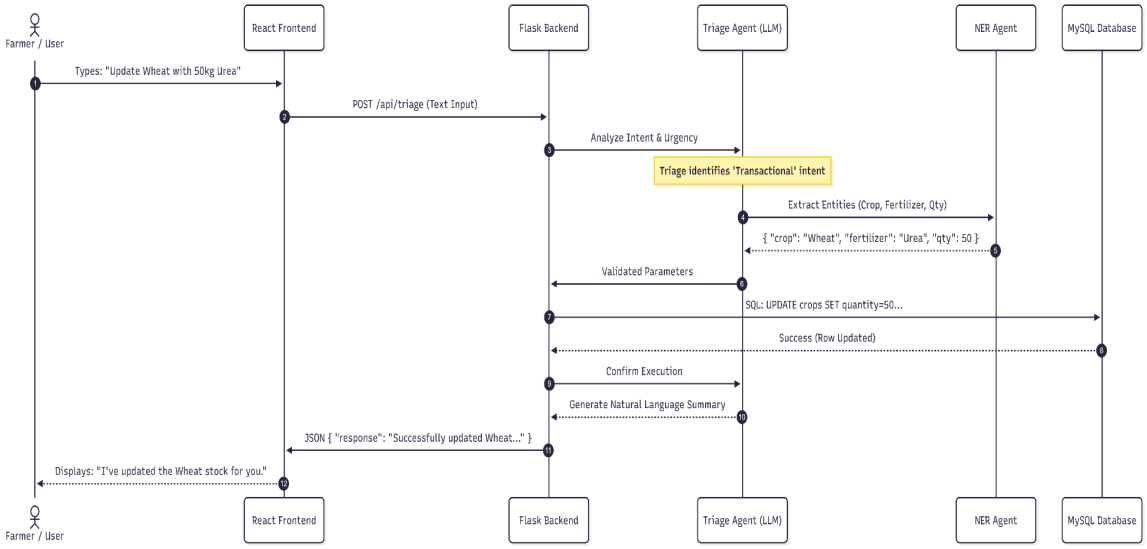
The architecture follows a **Decoupled Agentic Loop** where the AI is the central decision-maker.

1. **Interaction Layer:** The user enters a command in the **React/Vite** frontend.
2. **Orchestration Layer:** The request hits the **Flask Backend**, where the **Triage Agent** classifies the intent (e.g., "Is this a data query or a new deployment?").

3. **Processing Layer (Sub-Agents):**
    - **SQL Agent:** If the intent is transactional, the agent writes and executes the query against the **MySQL** container.
    - **Expert Agent (RAG):** If the intent is informational, the agent retrieves context from **ChromaDB**.



4. **Execution Layer:** The system performs the action (e.g., updating the crops table) and generates a natural language summary.
5. **Delivery Layer:** The result is pushed back to the UI in real-time.



2.3Tools & Technologies Used

AI & Agentic Frameworks

- **LLM Engine:** Gemini 1.5 Flash / OpenAI GPT-4o (for agent reasoning).
- **Vector Database: ChromaDB** (for high-dimensional agricultural data embeddings).
- **NER Agent:** Custom Python-based logic for entity extraction.

Infrastructure & DevOps

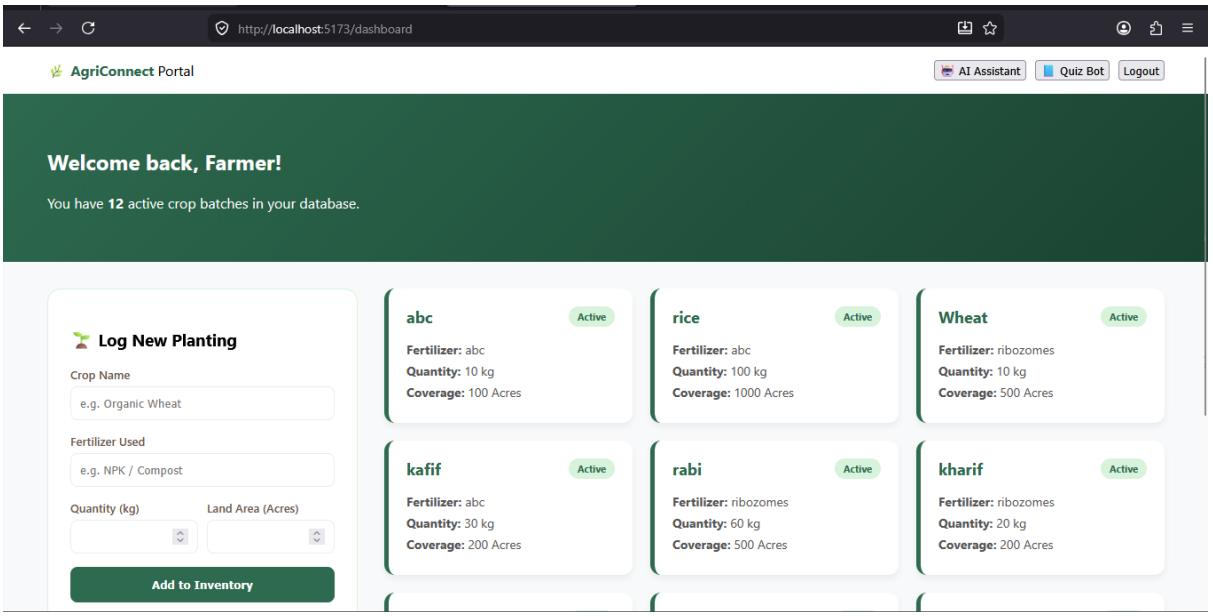
- **Containerization: Docker & Docker Compose** (for managing iac-backend and iac-db services).
- **Cloud Platform: AWS EC2** (for hosting the provisioning system).

Full-Stack Development

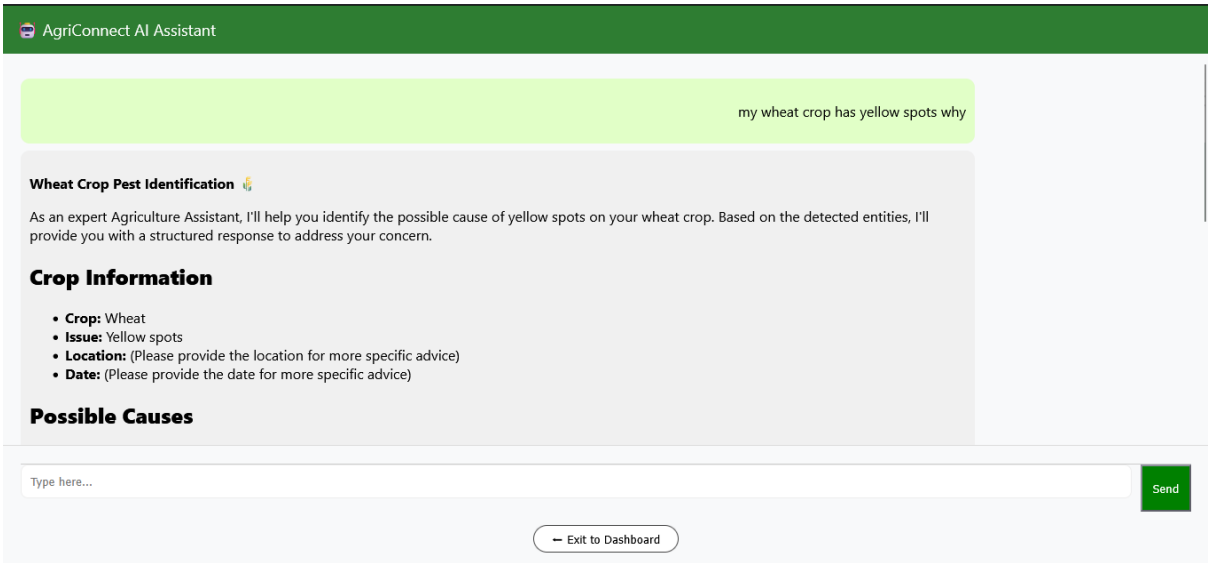
- **Frontend: React.js** with Vite.
- **Backend: Python Flask** (serving as the Agentic Orchestrator).

3 Result and Output

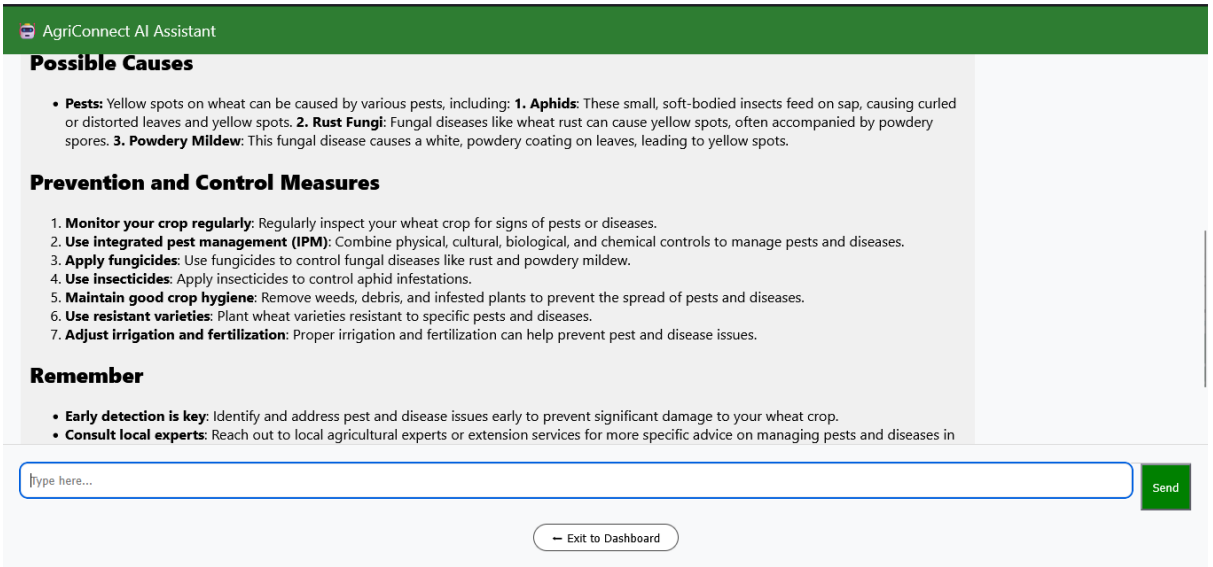
3.1 Screenshots / Outputs



Dashboard Page



AI Assistant



- **Agentic Triage Logs:** Evidence of the backend console showing the AI correctly identifying intent (e.g., "Intent: Database_Update" when a user types "Set rice quantity to 500").
- **Database State:** Screenshots of the `mysql` shell showing the `crops` table successfully populated with values extracted by the **NER Agent**.
- **Frontend Assistant Interaction:** Captures of the React chat interface where the AI provides a natural language summary of a database action.

3.2 Reports / Dashboards / Models

- **Inventory Dashboard:** A real-time data visualization within the React frontend that reflects the current state of the `agriculture_db`, updating automatically when the AI agent executes a command.
- **NER (Named Entity Recognition) Model:** The system utilizes a specialized **Transformer-based LLM** (e.g., Gemini 1.5 Flash) as a zero-shot NER model to parse agricultural entities without requiring a manually labeled training dataset.
- **Connectivity Model:** A microservices model where the `iac-backend` and `iac-db` communicate over a dedicated Docker bridge network, ensuring data integrity and low latency.

3.3 Key Outcomes

- **Elimination of Technical Latency:** The **Triage Agent** reduced the time from "user intent" to "database execution" by automating the query generation process, removing the need for manual data entry.

- **High Extraction Accuracy:** The **NER Agent** successfully extracted entities (Crop Name, Fertilizer, Quantity) with high precision, even from conversational and informal "Hinglish" inputs.
- **Seamless IaC Provisioning:** Demonstrated the ability to manage a containerized stack through a single orchestrator, proving that Agentic AI can effectively handle **Infrastructure as Code** tasks.
- **Contextual Intelligence:** By integrating **ChromaDB**, the system moved beyond simple data storage to provide "Expert Advice," such as suggesting the best fertilizer based on the specific "Area" stored in the database.

4. Conclusion

4.1 Brief Conclusion

The **AgriConnect IaC Provisioning System** successfully demonstrates the integration of **Agentic AI** with modern DevOps practices to solve real-world agricultural data challenges. By developing a multi-agent architecture, the project bridges the gap between complex cloud infrastructure and non-technical end-users.

Key Project Achievements:

- **Autonomous Orchestration:** Successfully implemented a **Triage Agent** that replaces static interfaces with a dynamic reasoning loop, allowing the system to interpret intent and route tasks autonomously.
- **Seamless Data Interaction:** Enabled "Zero-UI" database management where the **NER (Named Entity Recognition) Agent** converts unstructured natural language into structured SQL commands to manage the agriculture_db.
- **Infrastructure as Code (IaC):** Leveraged **Docker and Docker Compose** to create a resilient, containerized environment that can be deployed at scale on **AWS EC2**.
- **Contextual Intelligence:** Combined structured data from **MySQL** with unstructured expert knowledge in **ChromaDB** using **RAG**, providing users with actionable insights rather than just raw data.

Key Learnings:

- **Agentic Frameworks:** Developed a deep understanding of the **Triage pattern**, learning that specialized sub-agents (SQL, RAG, and Provisioning) provide higher accuracy and lower latency than a single generalized LLM.
- **DevOps Synergy:** Gained practical experience in container networking, data persistence through **Named Volumes**, and the lifecycle management of a microservices-based AI application.

- **Problem-Solving:** Learned how to handle the challenges of parsing informal language (like Hinglish) into precise technical actions, a critical skill for localized digital solutions.

In summary, this project serves as a robust prototype for future agricultural technology, proving that **Agentic AI** can act as a powerful operator for both data management and infrastructure provisioning.

5. Future Scope & Enhancements

- **Voice-Activated Agents:** Integrating Speech-to-Text so farmers can interact with the system using voice commands in regional languages.
- **Predictive Analytics:** Using the historical data in `agriculture_db` to train a regression model that predicts future crop yields based on current fertilizer levels.
- **Multi-Cloud Deployment:** Expanding the IaC capabilities to provision resources across **AWS, Azure, and Google Cloud** simultaneously based on cost-optimization logic.